



DESCRIPCIÓN DEL DISEÑO

DISEÑO DE APLICACIONES - 2

<https://github.com/IngSoft-DA2/274724-281190-250230>



INTEGRANTES:

Agustin Do Canto - 250230

Juan Ignacio Díaz - 274724

Santiago Remedi - 281190

INDICE

Errores conocidos.....	2
Deudas técnicas.....	2
Diagrama de paquetes general.....	4
Paquetes principales.....	5
Namespaces / Carpetas dentro de paquetes.....	7
WebApi.....	7
Descripción de herencias utilizadas.....	7
Modelo de tablas de la base de datos.....	8
Diagramas de secuencia.....	9
Justificación del diseño.....	10
Mecanismos de inyección de dependencias utilizados.....	10
Ciclo de vida utilizado en dependencias de la aplicación.....	10
Cumplimiento de principios de diseño.....	10
Patrón Adapter.....	10
SRP (Single Responsibility Principle).....	11
OCP (Open-Closed Principle).....	11
ISP (Interface Segregation Principle).....	12
DIP (Dependency Inversion Príncipe).....	12
Decisiones de diseño propias.....	12
Descripción del mecanismo de acceso a datos utilizado.....	15
Descripción del manejo de excepciones.....	15
Diagrama de componentes.....	17

Descripción general del trabajo y errores

Alcance

La solución realizada permite el registro de usuarios y de hogares y dispositivos inteligentes en estos, pudiendo ver su estado en todo momento y generando notificaciones si el dispositivo realiza alguna acción en concreto.

La aplicación permite a dueños de hogar registrarse y autenticarse, registrar su hogar en la plataforma y asociar dispositivos a su hogar, así viendo su estado y recibiendo notificaciones de las acciones de los dispositivos. Pueden ser dueños de más de un hogar a la vez, y pueden ser agregados a otros hogares como miembros, en donde el dueño del hogar podrá elegir que permisos tiene, que incluyen listado de miembros del hogar, listado de dispositivos asociados y la capacidad de recibir notificaciones.

La aplicación también tendrá usuarios administradores, que serán los encargados de mantener ciertos aspectos de la aplicación. Podrán crear cuentas de administrador, eliminar cuentas de administrador, listar los usuarios que se encuentran registrados, crear cuentas de dueño de empresa incompletas y listar todas las empresas registradas.

Los dueños de empresa podrán activar sus cuentas registrando la empresa de la que son dueños. una vez terminado este proceso, podrán registrar dispositivos que vende la empresa en la aplicación. Hoy, se soportan dos tipos de dispositivos: cámaras y sensores. Estos dispositivos son los que los dueños de hogar asociarán a su hogar. Una vez esta asociación se realiza, se genera un HardwareId, que representa al dispositivo físico en el hogar.

Errores conocidos

Deudas técnicas

- No fue implementada una funcionalidad para encender o apagar HomeDevices, esto provoca que a no ser que su ConnectionState se ponga en 1/true en la base de datos, no se generarán notificaciones. Esto se podría implementar teniendo dos endpoints con ruta /device/{hardwareId}/on y /device/{hardwareId}/off, en estos dos se establecería el estado de conexión del dispositivo en On y Off respectivamente. El verbo utilizado para estos sería PUT o PATCH.
- No se implementó el filtrado para las notificaciones de un usuario, los parámetros se reciben correctamente pero no son utilizados en ninguna función que los recibe, esto también es un error de diseño.
- No se implementó el filtrado de autorización por permisos específicos del hogar, por lo que no es necesario tener permisos específicos para ejecutar acciones

específicas del hogar como recibir notificaciones, listar dispositivos del hogar o listar miembros del hogar. A pesar de esto, los permisos existen en la base de datos.

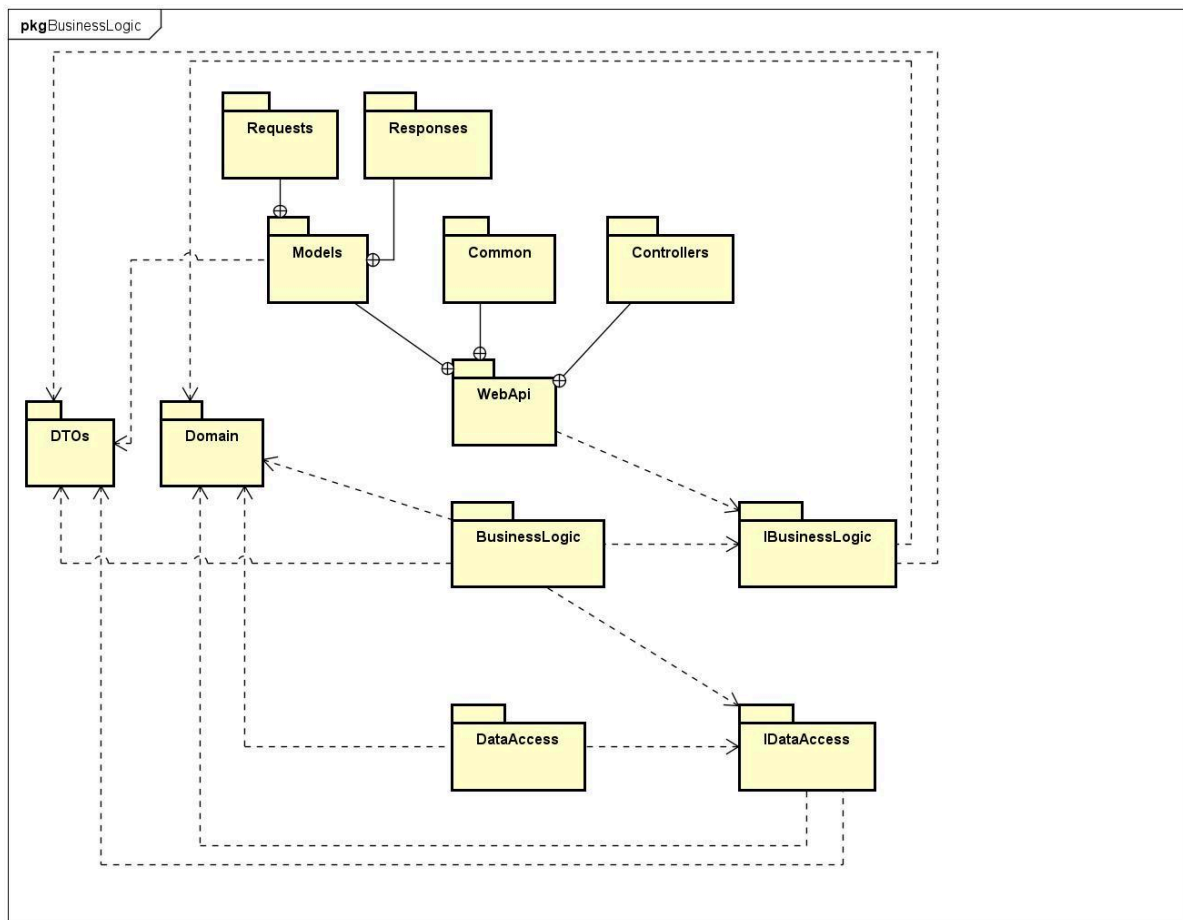
Bugs

- Las notificaciones generadas por las acciones de un sensor no se crean para cada miembro que pertenece al hogar, sino que se crean para un solo miembro. Además, al ser creadas, no se agrega correctamente el usuario al que la notificación se dirige. Esto se debe a que, por fallas de sincronización, tenemos dos implementaciones distintas para la generación de notificaciones. La generación de notificaciones para cámaras se realiza correctamente, pero como para los sensores utilizamos una implementación distinta que no genera una notificación por miembro del hogar en donde se ubica el dispositivo.
- Al intentar asociar una compañía con un Company Owner que ya tiene una asociada, se lanza un error correctamente pero la compañía se crea de todas maneras, sin un Company Owner que la tenga asociada. Esto sucede porque la validación de que el usuario ya tenga una compañía asociada sucede luego de la creación de esta, por lo que se crea sin que un usuario la asocie.
- Cuando un sensor genera notificaciones para los miembros de un hogar, no se genera una notificación para el dueño de hogar. Esto se debe a un error al implementar el método que se encarga de generar las notificaciones, genera uno por miembro del hogar pero no genera una para el dueño del hogar.
- Al crear un usuario dueño de empresa con un correo ya registrado perteneciente a un usuario dueño de hogar, se obtiene un error inesperado (500)

Errores de diseño

- En un principio, hicimos el diagrama de clases de dominio pensando en que HomeOwner debería tener una lista de notificaciones para facilitar el manejo de listado de todas las notificaciones de un miembro de hogar. Sin embargo, en nuestra implementación, la clase HomeOwner no tiene su propia lista de notificaciones, por lo que necesitamos hacer consultas al repositorio constantemente para ir obteniendo y actualizando las notificaciones.
- Home tiene un atributo innecesario. Presenta un atributo OwnerEmail y además tiene un HomeOwner. Lo ideal habría sido que tuviera únicamente un atributo de tipo HomeOwner, ya que a partir de él se podría obtener el email, por lo que nuestra implementación es redundante.

Diagrama de paquetes general



Justificación de arquitectura

Utilizamos una arquitectura en capas o *layered architecture*. La elegimos porque separa las responsabilidades de la aplicación de forma clara, cada capa tiene una responsabilidad específica. Esto permite modularidad y permite que al realizar cambios en una capa en concreto no se afecten las demás. Gracias a esta modularidad se puede cambiar la implementación de cualquiera de las 3 capas principales (presentación, negocio, acceso a datos) sin impactar a las otras 2. Justamente por esta razón también esta arquitectura aporta reutilización de código, se pueden utilizar estos módulos en otros contextos. Además, al estar organizado en capas, el código tiene una estructura más definida, lo que facilita su mantenimiento a futuro. También, una de las razones por las que la elegimos es la familiaridad que tenemos, la conocemos.

Pensamos en utilizar una *vertical slice architecture* pero nos decidimos por la alternativa ya que esta arquitectura tiene mayor complejidad inicial a la hora de implementar funcionalidades, particularmente para nosotros que venimos trabajando con la arquitectura en capas. Además, en comparación a la alternativa, esta arquitectura es más difícil de escalar, aunque a pesar de todo esto reconocemos sus ventajas (alta cohesión y reducción de acoplamiento en comparación a la *layered architecture*)

Descripción de responsabilidades de paquetes

Como se puede ver en el diagrama, tenemos 3 paquetes principales, cada uno representado por un proyecto. Estos son BusinessLogic, DataAccess y WebApi. Cada uno de estos paquetes representa una capa del modelo de 3 capas; BusinessLogic representando la capa de negocio, DataAccess la capa de acceso a datos y WebApi representando la capa de presentación. Luego, tenemos paquetes de interfaces (IBusinessLogic y IDataAccess) cuyo propósito es ayudar a cumplir DIP (se profundiza en esto más adelante). También tenemos paquetes para tanto las entidades de dominio como para DTOs. Por último, tenemos paquetes dentro del paquete WebApi que agrupan componentes por su tipo; Common para filters tanto de autenticación como excepciones, Models para DTOs de requests y de respuestas a estas, y Controllers para almacenar las clases encargadas de responder a las peticiones que se reciban.

Paquetes principales

- WebApi: representa a la capa de presentación, contiene los controladores que procesan requests HTTP y los filtros tanto de excepciones como de autenticación.
- BusinessLogic: representa a la capa de negocio, contiene las entidades de dominio y los servicios necesarios para manipular el dominio para que la aplicación cumpla con los requerimientos.
- DataAccess: representa a la capa de acceso a datos. Es el medio por el que BusinessLogic interactúa con EntityFramework y su clase DbContext, el medio por el que las entidades manipuladas por los servicios se almacenan.
- Domain: contiene las entidades de dominio que serán utilizadas por toda la aplicación, y manipuladas por la capa de negocio.
- DTO's: contiene los modelos (Data Transfer Objects) a través de los cuales se recibirá y mostrará información específica dependiendo del contexto al cliente
- IBusinessLogic: contiene las abstracciones implementadas por los servicios ubicados en el proyecto BusinessLogic (capa de negocio)
- IDataAccess: contiene las abstracciones implementadas por los repositorios concretos ubicados en el proyecto DataAccess (capa de acceso a datos)

Decidimos realizar los diagramas de clases mostrados a continuación ya que son los que nos parece que tienen más valor para mostrar, estos serían: Domain (para poder observar como las entidades de dominio se relacionan entre sí y su cardinalidad), BusinessLogic (para poder ver más claramente la dependencia de los servicios que utilizamos) y IDataAccess (para mostrar cómo segregamos las interfaces y qué relaciones de herencia se pueden encontrar, nos saltamos los métodos para evitar dar detalle innecesario). Realizar diagramas de clases del paquete DTO por ejemplo, o de Controllers no es tan valioso ya que las entidades de los paquetes no se relacionan entre ellas.

Diagramas de clases de paquetes principales

Diagrama de clases de Domain

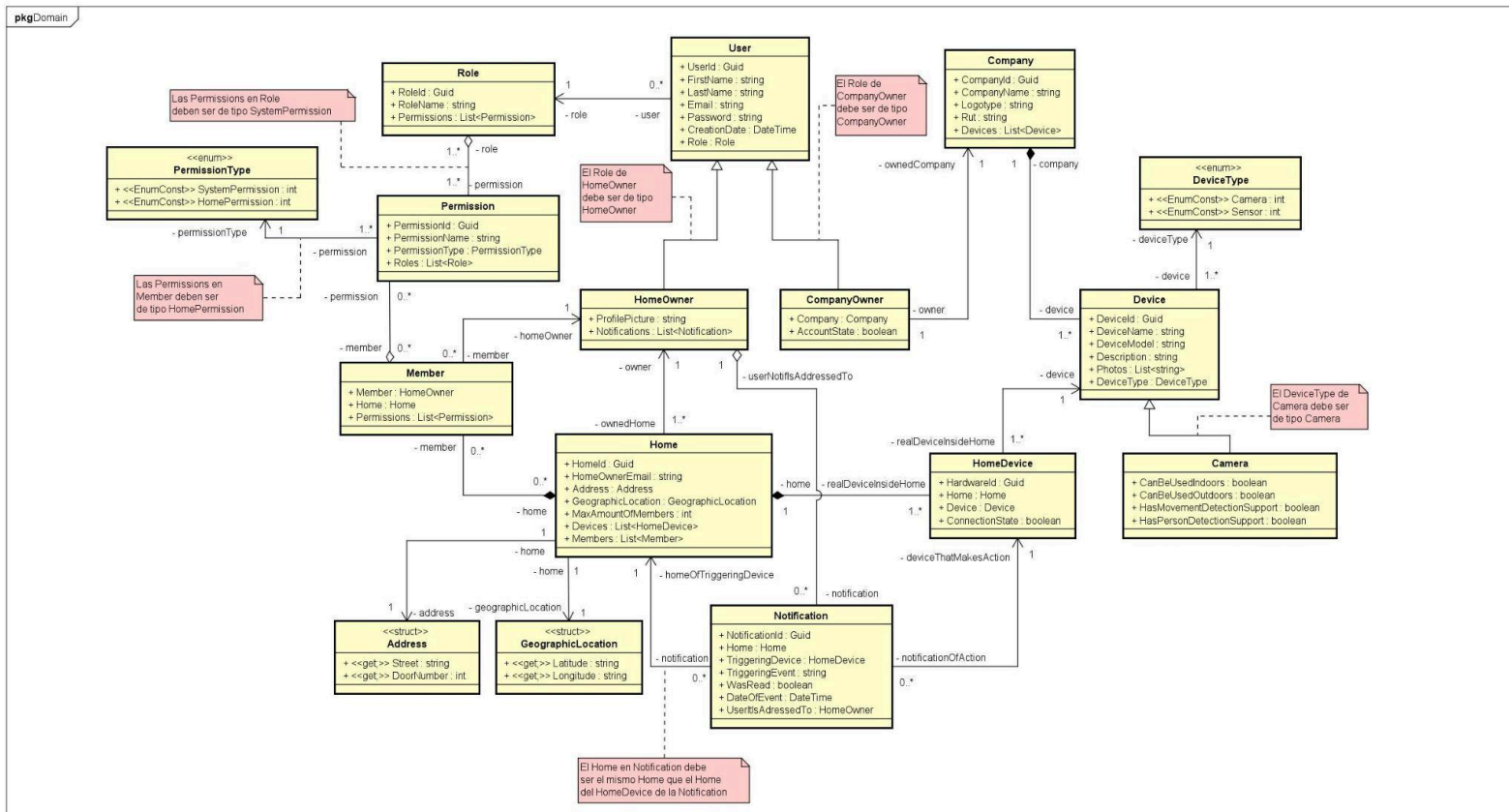


Diagrama de clases de BusinessLogic

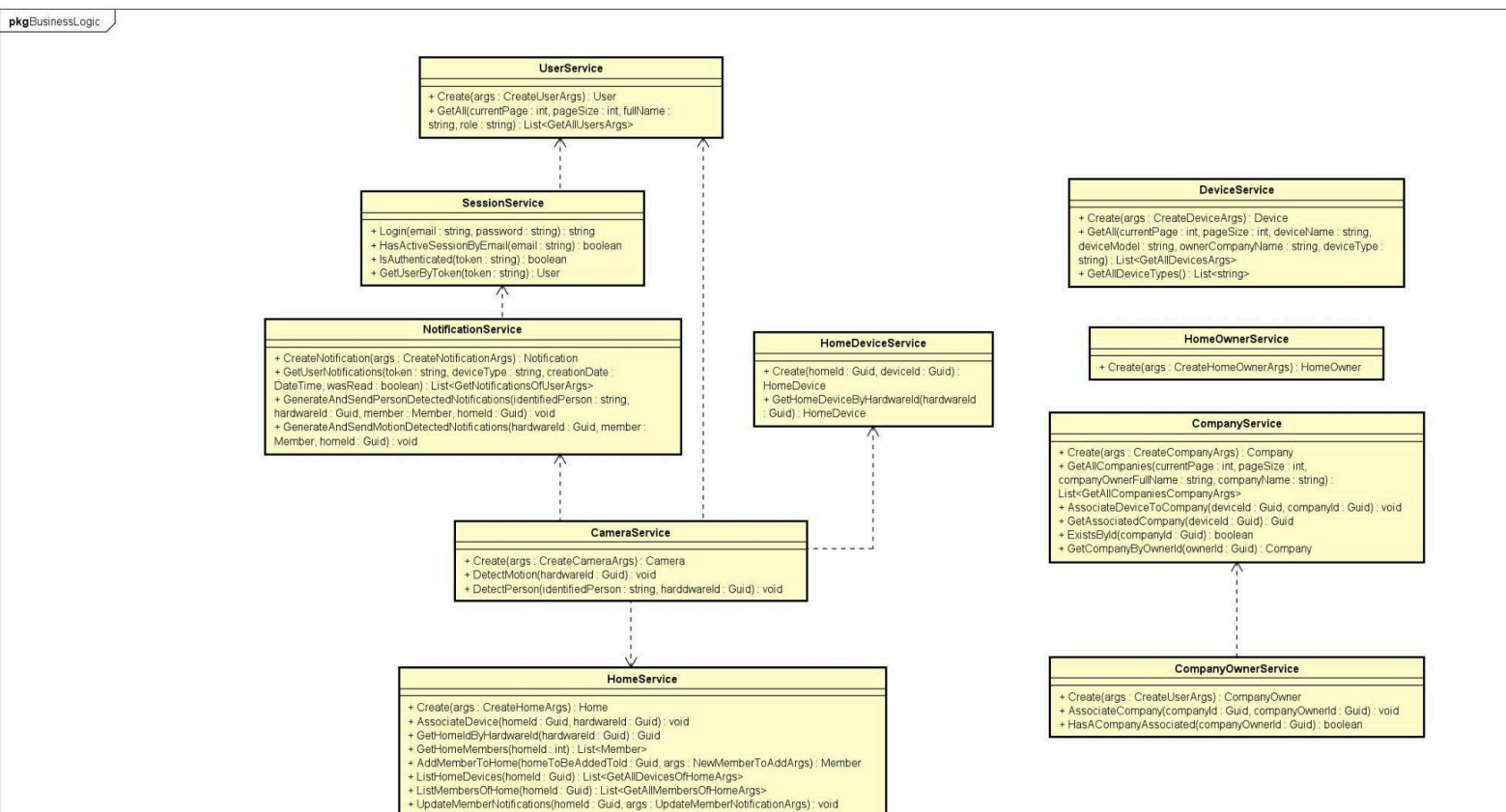
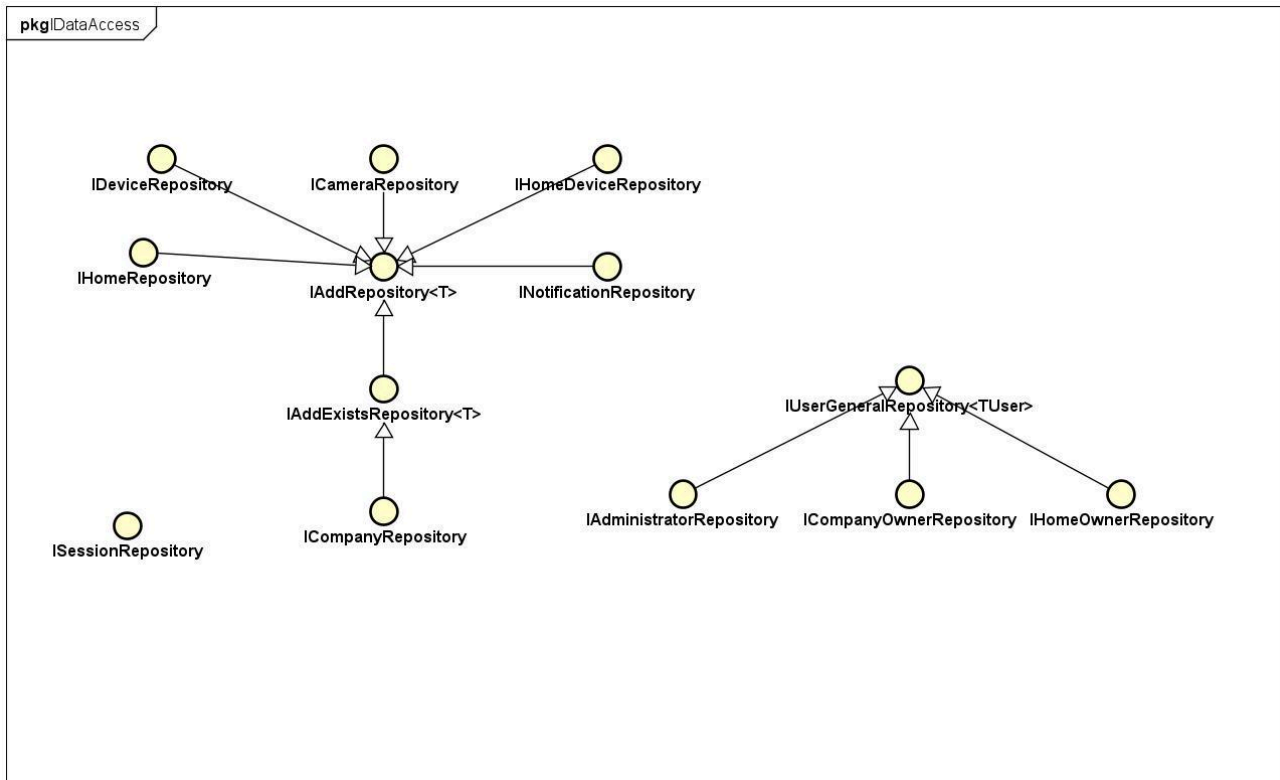


Diagrama de clases de IDataAccess



Namespaces / Carpetas dentro de paquetes

WebApi

- **Controllers:** contiene todos los controladores de la aplicación, los cuales se encargan de procesar requests.
- **Common:** contiene los filtros que se ejecutan antes de los controladores al recibir requests, estos son: filtro de autenticación, de autorización y de excepciones.
- **Models:** contiene los modelos que representan los argumentos de entrada de requests y lo que estas terminarán respondiendo.
- **Models/Requests:** contiene los modelos que contienen los datos enviados en las requests.
- **Models/Responses:** contiene los modelos que contienen los datos que se envían como respuesta de una request.

Descripción de herencias utilizadas

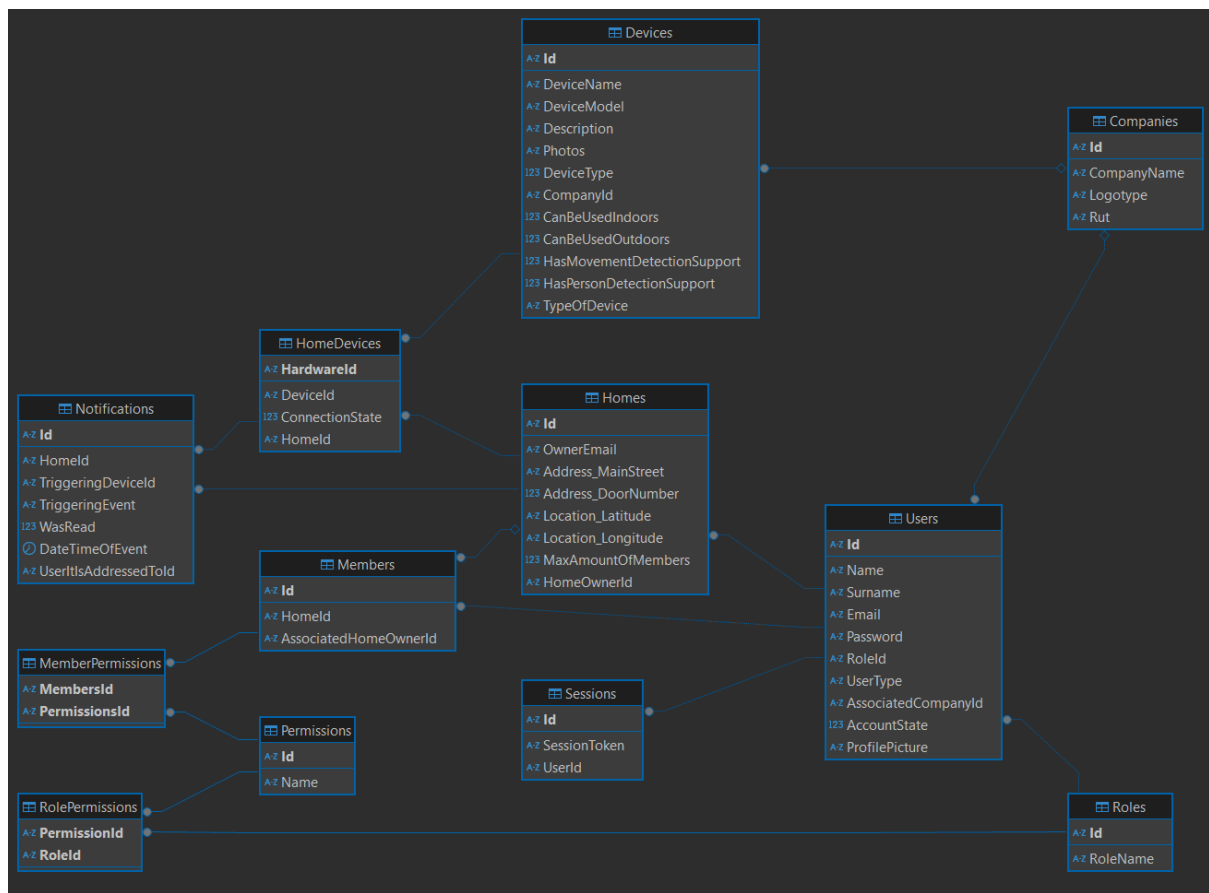
En nuestra solución utilizamos dos herencias:

- **User, CompanyOwner y HomeOwner:** decidimos utilizar una herencia para representar a los diferentes tipos de usuarios ya que los tres tipos (los dos mencionados y Administrator) tienen múltiples atributos en común, y cada tipo de

usuario tiene relación con diferentes entidades. Por otro lado, no tenemos una clase Administrator que hereda de user ya que, si la hacemos, sería una clase vacía, ya que no tiene comportamiento único y comparte atributos con User, por lo que tendríamos una clase vacía sin propósito.

- Device y Camera: para modelar los dispositivos decidimos utilizar una herencia ya que ambos tipos de dispositivos (Camera y Sensor) tienen múltiples atributos en común, pero como no tienen comportamiento único no incluimos una clase Sensor (similar a User y Administrator)

Modelo de tablas de la base de datos



Al realizar las migraciones que fuimos construyendo con EF Core obtuvimos el modelo mostrado en la imagen. Como se puede ver en el diagrama, hay un error en la tabla Notifications, ya que UserItIsAddressedToId debería ser una FK hacia la tabla Users en el atributo Id de esta, esto no es así por fallas al configurar el modelo en el método OnModelCreating de nuestro DbContext

Diagramas de secuencia

Al pensar en qué funcionalidades podríamos diagramar, nos decidimos por la creación de empresa, y la eliminación de administradores ya que son requerimientos con (*) y tienen un flujo interesante como para mostrar en un diagrama de interacción. En estos diagramas se representa el flujo “correcto”

Diagrama de secuencia de Associate Company

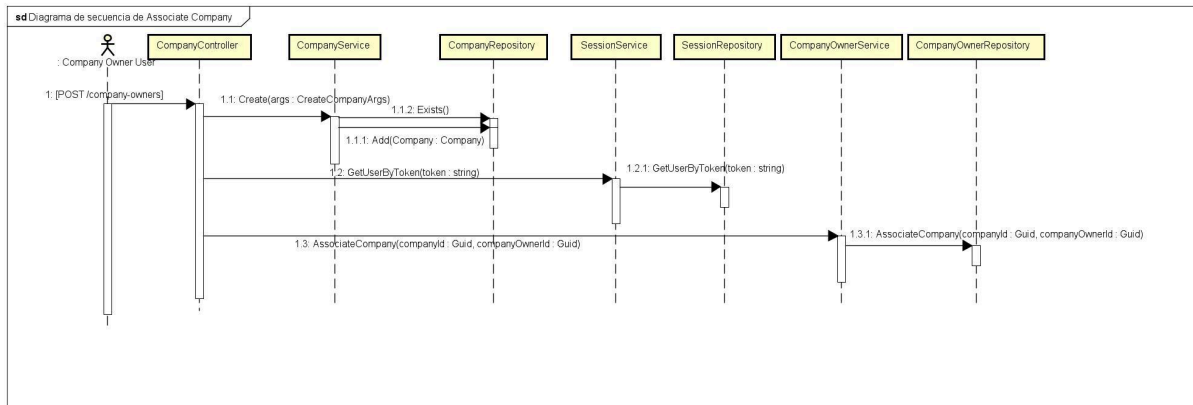
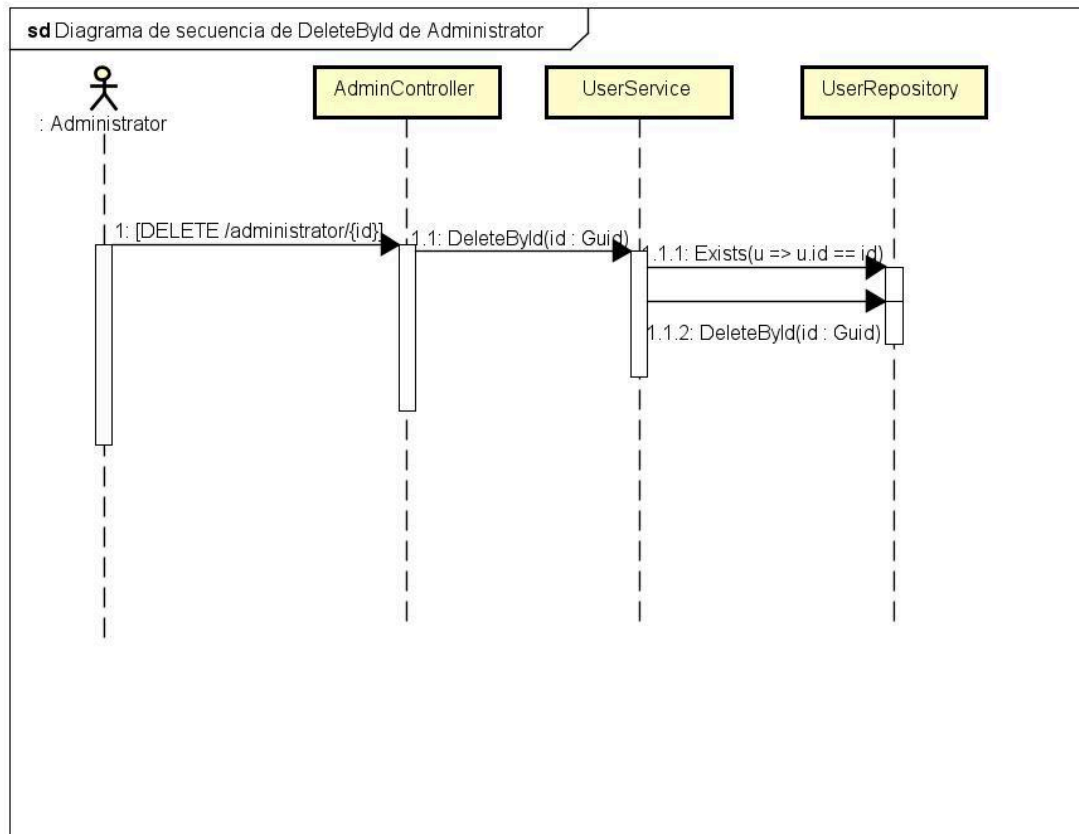


Diagrama de secuencia de DeleteByld de Administrator



Justificación del diseño

Mecanismos de inyección de dependencias utilizados

Ninguna de las clases que tenemos instancian sus dependencias, sino que todas son recibidas por parámetro. Esto es porque no es responsabilidad de las clases crear sus dependencias.

Por parámetro se recibirán interfaces de las dependencias, en vez de recibir una implementación concreta o instanciarlas en el constructor, lo que nos genera poco acoplamiento (la dependencia es hacia una interfaz y no hacia otras clases o paquetes), permite mockear y aporta modularidad y extensibilidad (resistente a cambios y a nuevas funcionalidades). Además, cumple con OCP (se puede extender la interfaz recibida como dependencia sin que impacte en la clase que la recibe) y SRP (por lo mencionado anteriormente).

Ciclo de vida utilizado en dependencias de la aplicación

Para inyectar los servicios en la app a la clase Program.cs, utilizamos el ciclo de vida scoped ya que este es el que permite que las dependencias de la aplicación vivan todo a lo largo de una request, lo que a su vez permite que las requests sean independientes, que también aporta consistencia.

Utilizar servicios scoped también ahorra problemas de mantener en memoria servicios más tiempo de lo que es necesario. No tiene sentido que un servicio creado para manejar una request particular siga existiendo después de que fue procesada, también ahorra problemas de concurrencia y de performance (que se podría reducir si se utilizara transient) y evita compartir el estado entre clientes cuando no se desea (lo cual singleton podría causar).

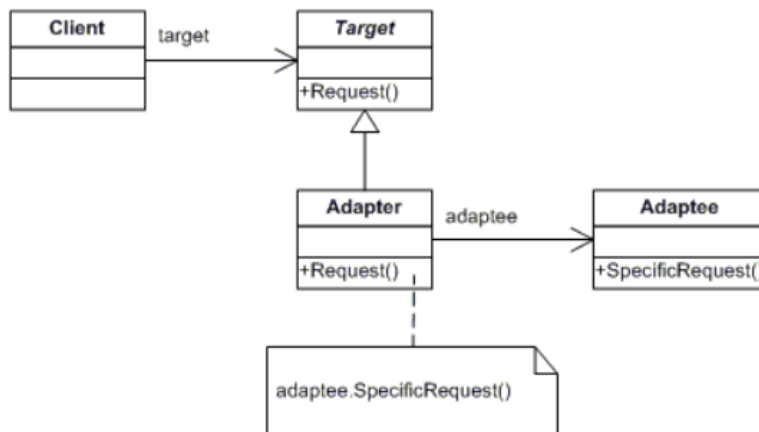
Cumplimiento de principios de diseño

Patrón Adapter

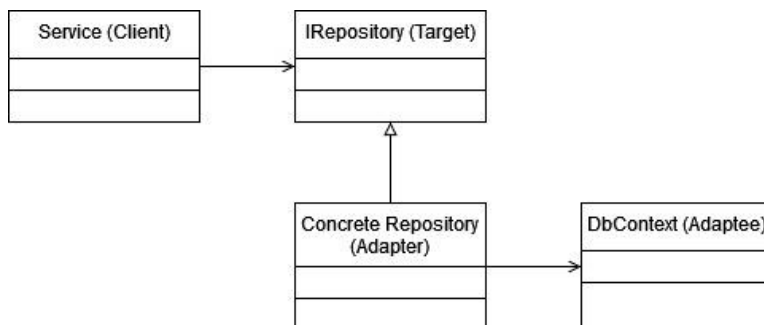
Utilizamos el patrón Adapter de la siguiente manera: el Client serían los servicios que se encuentran en el paquete BusinessLogic, El Adapter serían los repositorios que se encuentran en DataAccess, y el Adaptee sería el paquete EntityFrameworkCore (más específicamente, la clase DbContext de este).

Utilizamos el patrón Adapter para que, si algo cambia en cómo funciona el paquete Entity Framework, para que no haya mayor impacto ni se rompa ningún servicio BusinessLogic, además, para evitar inyectar un DbContext en BusinessLogic directamente, lo que podría incumplir el DIP.

Patrón genérico



Utilización concreta



SRP (Single Responsibility Principle)

Cumplimos SRP separando las responsabilidades de las distintas clases para asegurarnos que no hagan cosas que “no deberían” hacer. Un ejemplo de esto es los servicios. Tenemos un servicio por entidad de dominio, y en el caso de que un servicio necesite realizar acciones que no son de esa entidad de dominio respectiva, se le inyecta otro servicio como dependencia para que este realice las acciones. También lo vemos en las validaciones de los argumentos de las clases de dominio. Previamente, teníamos las validaciones en los respectivos servicios, pero decidimos moverlos al DTO que se transforma en el objeto de negocio, ya que el DTO es el que conoce los argumentos primero que nadie más.

OCP (Open-Closed Principle)

Una de las razones por las que decidimos tener herencias para **User** y para **Device** es para cumplir OCP, esto nos permite, si se agregan nuevos tipos de usuarios o dispositivos, no modificar el código existente.

También tuvimos el principio en cuenta al modelar los roles y los permisos, ya que a la hora de agregar nuevos roles o permisos en el futuro, simplemente se debe insertar una tupla en las tablas correspondientes, es decir, agregar sin modificar lo existente.

En relación a los endpoints, tuvimos en cuenta el principio al crear los endpoints que generan notificaciones cuando un dispositivo realiza una acción en concreto, ya que decidimos tener un endpoint por acción, lo cual aumenta ligeramente la complejidad pero decidimos que cumplir OCP sería más importante en este caso, ya que si el día de mañana se presenta una acción nueva, simplemente se agrega un endpoint.

ISP (Interface Segregation Principle)

Cumplimos este principio de diseño al utilizar interfaces padre que les proveen a las interfaces que luego serán implementadas por clases concretas solamente las funciones que se utilizarán en estas clases concretas. Esto principalmente se puede ver en como manejamos la abstracción de los repositorios, para los cuales tenemos dos interfaces genéricas que luego serán heredadas por interfaces con funcionalidades más concretas (`IAddRepository<T>` y `IAddExistsRepository<T>`, en donde la segunda hereda de la primera). Además, tenemos una interfaz que es utilizada por los 3 tipos de usuario existentes actualmente, ya que comparten cierto comportamiento en común (`IUserGeneralRepository<TUser>`).

DIP (Dependency Inversion Principle)

Nos aseguramos de cumplir DIP logrando que ningún módulo o paquete de alto nivel dependa de uno de bajo nivel, sino que se dependa de abstracciones lo más estables posible.

Para esto, decidimos que tanto los controladores como los servicios deben depender de interfaces. Los controladores dependerán de interfaces que serán implementadas por los servicios, y los servicios dependerán de interfaces que serán implementadas por los repositorios. De esta forma también se logra que los módulos sean intercambiables, por lo que si se quiere utilizar otro repositorio u otra implementación de los servicios, se puede hacer sin hacer que la aplicación se rompa.

Decisiones de diseño propias

Roles y permisos

Para modelar los distintos roles y permisos, utilizamos una clase `Role` y una clase `Permission` respectivamente. Suponemos que los roles y los permisos no cambian mucho por lo que leímos de la letra, por eso, decidimos “hardcodear” los roles y los permisos, para que ya al levantarse la base de datos estos estén ingresados.

Además, tenemos dos tipos de permisos; los permisos que les permiten a los usuarios realizar acciones (por ejemplo, crear una `Company`, eliminar usuarios, etc) y los permisos que son de un hogar específico, que pertenecen a los miembros (por ejemplo, listar los `Devices` de un hogar específico).

Estos permisos serán diferenciados por un `PermissionType` (enum). Los permisos relacionados a los roles serían de tipo `SystemPermission`, y los relacionados a los hogares

serían de tipo HomePermission. Utilizamos un enum para diferenciarlos por lo mencionado anteriormente, que son cosas que no cambian seguido, por lo que decidimos priorizar la simplicidad antes que la extensibilidad.

Como todos los permisos requieren tener un rol, creamos un rol específico para agrupar los permisos que ningún usuario podrá tener.

Para decidir qué permisos serían necesarios, tomamos un permiso por acción que un usuario puede realizar. Esto puede ser alta, baja, modificación, listado, etc.

Usuarios y tipos de usuarios

Nuestra justificación para usar una herencia para los usuarios fue mayormente explicada en la sección de “Descripción de herencias utilizadas”, pero, también aquí podemos agregar que contemplamos alternativas, pero ninguna era mejor que la herencia dada la naturaleza de los requerimientos.

A su vez contemplamos que la clase User sea una clase abstracta, y que de ella hereden Administrator, HomeOwner y CompanyOwner, pero decidimos que, dado que la supuesta clase Administrator no tendría diferencia con su padre User, terminaría siendo una clase vacía, lo cual no sería correcto ya que su estado (atributos) y comportamiento (métodos) estarían en su padre, por lo que eso nos llevó a hacer User una clase concreta y que sus hijos sean HomeOwner y CompanyOwner. Al crear estos usuarios, cada tipo de usuario tendría un rol (explicitado por la letra/foro), por lo que les asignamos los “hardcodeados” al crear instancias de las clases. En cuanto a atributos, tomamos el correo del usuario como único.

Compañías

En esta entidad no hubo muchas decisiones para tomar sobre atributos y cosas del estilo. Asumimos que el rut de una empresa tiene que ser único, y que la relación entre una compañía y un dueño de compañía es uno a uno. También, una vez que un dueño de compañía asocia una empresa a su cuenta, no puede asociar una nueva.

Hogares y miembros

Para los hogares no tomamos decisiones de diseño particulares, excepto por sus miembros y tanto su dirección como su ubicación geográfica. Decidimos que estos últimos dos deberían ser Value Types (tipos identificados por sus valores y no por su espacio en memoria), pero sin la restricción de ser únicos, es decir, puede haber 2 hogares con la misma dirección. Para esto, los tipos deberían ser structs, pero por restricciones de EF, tuvimos que hacer que sean clases.

Para representar los miembros de un hogar, utilizamos una clase Member, que contiene a un HomeOwner y una lista de permisos.

De los requerimientos inferimos que un HomeOwner puede ser miembro de varios hogares sin ser el dueño, y en cada hogar puede tener distintos permisos, por eso utilizamos la clase Member. Luego, un hogar tendrá una lista de miembros, cada uno con sus permisos, de esta forma modelamos de forma muy cercana a la realidad.

Previamente teníamos un servicio y un repositorio para la entidad Member pero nos dimos cuenta que, como la entidad solo existe dentro de home, sería buena idea que la generación de estos suceda en HomeService, ya que este es el experto en los datos del miembro.

Dispositivos y dispositivos de hogar

De la herencia de Device y Camera, se habló en el punto “Descripción de herencias utilizadas”, y es un caso similar al de “Usuarios y tipos de usuarios”.

Consideramos tener una clase abstracta pero para simplificar utilizamos una concreta, y como Sensor no tiene atributos ni comportamiento único (a diferencia de Camera), decidimos no tener una clase vacía. Para esta herencia decidimos tener un enum representando los tipos de dispositivos (DeviceType) ya que la letra explícitamente indica que estos no cambian seguido, por lo que no tuvimos que preocuparnos por la extensibilidad. Las instancias de Camera serán de tipo Camera y las instancias de Device serán de tipo Sensor.

Para representar el dispositivo que se encuentra en un hogar (al agregar un dispositivo al hogar se genera un HardwareId y un estado de conexión) utilizamos una clase HomeDevice, que tiene un Device, esta Id y su correspondiente estado de conexión. Un hogar tendrá una lista de estos para representar los dispositivos físicos que se pueden hallar en la casa física.

Paginación y filtrado

Pensamos en implementar estas funcionalidades en la capa de negocio, pero al final decidimos implementar la paginación y el filtrado de datos en la capa de acceso a datos ya que esta capa es la más cercana a las entidades con todos sus datos, por lo que desde esta es más sencillo e intuitivo formar los datos que serán devueltos al cliente, incluyendo el filtrado y la paginación. Esto se puede ver, por ejemplo, en el método GetAll de la clase CompanyRepository, en este, aparte de datos de la entidad Company, se debe mostrar información de la entidad relacionada CompanyOwner, por lo tanto para devolver los datos completos se debe acceder al repositorio de esta entidad, que está “a mano” en todos los repositorios debido a que tienen acceso al DbContext de la aplicación.

Sabemos que al tener estas funcionalidades en la capa de negocio, aliviarnos al repositorio de tener ciertas responsabilidades que no son específicamente de él, pero en este caso valoramos más la cercanía a los datos que tienen los repositorios.

Decisiones generales

Hay cierto comportamiento que debería estar en los servicios que está o en los controladores o en los repositorios. En la mayoría de estos casos lo decidimos así ya que cualquiera de estas dos clases tenía más cercanía o “facilidad de acceso” a los datos que el servicio mismo, por lo que la facilidad de implementación es una ventaja considerable. Por otro lado, sabemos que esto puede hacer el cumplimiento de SRP en nuestro trabajo más débil.

Para marcar las notificaciones de un usuario como leídas, decidimos tomar que al ver sus notificaciones (notificaciones del usuario) estas se marcarán como leídas automáticamente. Tuvimos en cuenta que el usuario pueda hacerlo manualmente pero optamos por esta opción para simplificar la implementación.

Descripción del mecanismo de acceso a datos utilizado

En términos de carga de entidades, decidimos utilizar Eager Loading ya que es el que se nos hizo más familiar de utilizarlo previamente, y nos da más control a la hora de la carga de entidades relacionadas, también nos evita el olvidar que entidades fueron cargadas (por ejemplo, con lazy loading) y la complejidad de utilizar explicit loading.

Para el diseño de repositorios, decidimos tener un repositorio por entidad de dominio, cada uno se encargaría de su respectiva entidad y sería inyectado a su respectivo servicio. Para esto utilizamos interfaces, y todos los repositorios concretos que utilizamos implementarán una interfaz. Estos repositorios se encargan de realizar operaciones sobre la base de datos a través de la clase DbContext de EF Core. Estas operaciones pueden ser agregar datos, eliminar datos, realizar consultas y modificar datos existentes.

Descripción del manejo de excepciones

Decidimos utilizar excepciones del sistema dado que varios flujos de entidades no relacionadas del sistema pueden llevar al mismo error, por lo que tener excepciones personalizadas nos llevaría a lanzar varias excepciones distintas por el mismo tipo de error.

Cada excepción se mapea con un mensaje de error HTTP, de tal manera que, al enviar una request, si se genera una de las excepciones utilizadas, se devuelve el código de error correspondiente.

También utilizamos filtros para evitar el uso de sentencias try-catch en los controllers. Los filtros se ejecutan cuando se genera una excepción no manejada en un método de un controller.

Decidimos utilizar un filtro global, que maneje todas las excepciones ya que llegamos a la conclusión de que construir múltiples filtros pequeños, posiblemente uno por request, a la larga se volvería entreverado, y tendríamos que configurarlos en cada controlador. Por otro lado, utilizar uno global alivia esto, pero también tiene sus desventajas; requiere mantenimiento ya que maneja todas las excepciones del sistema, si en el futuro utilizamos un tipo nuevo tendremos que agregarlo al filtro.

Excepciones del sistema utilizadas

- `ArgumentNullException`: utilizada para notificar que un dato requerido es nulo o vacío (Representa 400 Bad Request)
- `KeyNotFoundException`: utilizada para notificar que al agregar o modificar un elemento, se le pasó un argumento que es clave, pero ningún elemento con la clave ingresada existe (Representa 404 Not Found)
- `FormatException`: utilizada para notificar que un argumento se envió con formato incorrecto (número negativo donde no se permite, un valor que debería estar en un enum no se pudo pasear, etc) (Representa 400 Bad Request)
- `InvalidOperationException`: utilizada para argumentos con claves repetidas ó ya existentes (representa 409 Conflict)
- Default (ninguna de las anteriores): 500 Server Error

Para el manejo de errores relacionados a la autorización y autenticación utilizamos un filtro global también, aquí aparecen dos errores, el de falta de permisos (403 forbidden) y el de falta de autorización válida (401 unauthorized). El primero se lanza cuando un usuario ingresa con autenticación válida pero no tiene permisos para realizar una acción, y el segundo se lanza cuando un usuario con una autenticación inválida.

Diagrama de componentes

